



PROGRAMAÇÃO FUNCIONAL (12026)

RELATÓRIO DO TRABALHO PRÁTICO Nº. 1 - Sistema de monitoramento - Cibersegurança.

Data: 24/09/2026

Professor: Dr. Wagner Igarashi

Discentes

R.A.	Nome
145100	Arthur Pedrinho de Paula
145092	Guilherme Henrique Viana Pichitelli Vitor



SUMÁRIO

1. Ciclo de projeto da função de validação ou classificação.....	3, 4
2. Ciclo de projeto da função que processa uma lista por recursão estrutural.....	5
3. Ciclo de projeto da função que utiliza funções auxiliares ou um plano de solução....	6, 7
4. Ciclo de projeto da função que produz uma nova lista.....	8
5. Ciclo do projeto da função que processe a estrutura autorreferente hierárquica.....	9,10



Função de validação ou classificação:

Análise:

Faça uma função que recebe um evento e devolve o mesmo evento, porém com a severidade atualizada de acordo com a quantidade de tentativas dele. A periculosidade de um evento segue a seguinte classificação:

Um evento com 0 tentativas tem periculosidade nula;

Um evento com menos que 3 tentativas tem periculosidade baixa;

Um evento com mais ou igual que 3 e menos que 5 tentativas tem periculosidade média;

Um evento com mais ou igual que 5 e menos que 10 tem periculosidade alta;

Um evento com mais ou igual a 10 tem periculosidade crítica.

Definição de tipos de dados:

A entrada será um evento, que será representado pelo tipo composto *Evento*. A saída será o mesmo evento com a severidade atualizada, que será representado pelo tipo composto *Evento*.

Especificação:

Recebe um evento e devolve o mesmo evento com a severidade atualizada de acordo com a quantidade de tentativas.

Implementação:

A implementação está no arquivo *programa_completo.gleam* das linhas 325 à 340.

Exemplos:

```
//EXEMPLOS F2: classifica_evento
pub fn classifica_evento_examples() {
  check.eq(classifica_evento(Evento(1, "Computador 1", Malware, Alta, 3, EmAnalise)), Evento(1, "Computador 1", Malware, Media, 3, EmAnalise))
  check.eq(classifica_evento(Evento(2, "Computador 2", AcessoSuspeito, Media, 1, Desconhecido)), Evento(2, "Computador 2", AcessoSuspeito, Baixa, 1, Desconhecido))
  check.eq(classifica_evento(Evento(3, "Computador 3", TentativaDeLogin, Baixa, 5, Resolvido)), Evento(3, "Computador 3", TentativaDeLogin, Alta, 5, Resolvido))
}
```



Revisão:

Inicialmente, considerou-se a possibilidade de utilizar a cláusula de guarda if dentro dos padrões. No entanto, optou-se por utilizar encadeamento de estruturas condicionais case para garantir a verificação exaustiva de todas as faixas de tentativas (0, menores que 3, menores que 5, menores que 10 e maiores ou iguais a 10). Os testes automatizados em *classifica_evento_examples* cobriram todos os casos limites, confirmando a corretude da função.



Função que processa uma lista por recursão estrutural:

Análise:

É necessário fazer uma função que encontre um evento em uma lista de eventos.

Definição dos tipos de dados:

Uma lista de eventos [List(Evento)] e um ID do tipo primitivo *Int*. A saída será o evento encontrado, representado pelo tipo composto *Evento* ou uma mensagem de erro, representada pelo tipo primitivo *String*.

Especificação:

Recebe uma lista de eventos e um ID, varre a lista de eventos em busca do evento com o ID informado, caso não encontre, retorna um erro.

Implementação:

A implementação está no arquivo *programa_completo.gleam* das linhas 107 à 120

Exemplos:

```
// Check da funcao: busca_evento_por_id
pub fn busca_evento_por_id_examples() {
  let evento1 =Evento(1, "Computador 1", Malware, Alta, 3, EmAnalise)
  let evento2 =Evento(2, "Computador 2", AcessoSuspeito, Media, 1, Desconhecido)
  let evento3 = Evento(3, "Computador 3", TentativaDeLogin, Baixa, 5, Resolvido)

  check.eq(busca_evento_por_id([evento1, evento2, evento3], 1),Ok(evento1))
  check.eq(busca_evento_por_id([evento1, evento2, evento3], 2),Ok(evento2))
  check.eq(busca_evento_por_id([evento1, evento2, evento3], 3),Ok(evento3))
  check.eq(busca_evento_por_id([evento1, evento2, evento3], 99),Error("Evento não encontrado"))
  check.eq(busca_evento_por_id([], 1),Error("Evento não encontrado"))
}
```

Revisão:

Inicialmente a *busca_evento_por_id* receberia um ativo, entretanto, pensando em possibilidades de receber listas de eventos não associadas de ativos de uma outra função, optou-se pela entrada ser diretamente uma lista de eventos.



Função que utiliza funções auxiliares ou um plano de solução:

Análise:

Faça uma função que receba uma lista de eventos e calcule a quantidade média de tentativas dessa lista.

Definição de tipos de dados:

A entrada será uma lista de eventos, que será representada pelo tipo composto **List(Evento)**. A saída será a quantidade média de tentativas, que será representada pelo tipo primitivo **Float**.

Especificação:

Recebe uma lista de **eventos** e devolve a quantidade média de tentativas.

Implementação:

A implementação está no arquivo *programa_completo.gleam* das linhas 358 à 362.

A das auxiliares está no mesmo arquivo das linhas 421 à 426 e 432 à 436.

Exemplos:

```
//EXEMPLOS F8
pub fn calcula_media_examples() {
  check.eq(calcula_media([Evento(1, "Computador 1", Malware, Alta, 3, EmAnalise), Evento(2, "Computador 2", AcessoSuspeito, Media, 1, Desconhecido)]), 2.0)
  check.eq(calcula_media([Evento(1, "Computador 1", Malware, Alta, 3, EmAnalise)]), 3.0)
  check.eq(calcula_media([]), 0.0)
}
```

```
pub fn conta_eventos_examples() {
  check.eq(conta_eventos([]), 0)
  let evento1 = Evento(1, "Computador 1", Malware, Alta, 3, EmAnalise)
  check.eq(conta_eventos([evento1]), 1)
  check.eq(conta_eventos([evento1, Evento(2, "Computador 2", AcessoSuspeito, Media, 1, Desconhecido)]), 2)
}
```

```
pub fn conta_tentativas_examples() {
  let evento1 = Evento(1, "Computador 1", Malware, Alta, 3, EmAnalise)
  let evento2 = Evento(2, "Computador 2", AcessoSuspeito, Media, 1, Desconhecido)
  let evento3 = Evento(3, "Computador 3", TentativaDeLogin, Baixa, 5, Resolvido)
  check.eq(conta_tentativas([evento1, evento2, evento3]), 9)
  check.eq(conta_tentativas([evento1]), 3)
  check.eq(conta_tentativas([]), 0)
}
```



Revisão:

O plano de solução adotado decompôs o problema principal em duas funções auxiliares puras com recursão estrutural simples (conta_eventos e conta_tentativas). Essa modularização facilitou a verificação independente dos subcomponentes. Na revisão, garantiu-se a conversão explícita dos inteiros para ponto flutuante via `int.to_float`, permitindo a divisão precisa por meio do operador `/`. Os testes automatizados confirmaram a precisão do cálculo em listas unitárias e com múltiplos elementos.



Função que produz uma nova lista:

Análise:

Faça uma função que recebe uma lista de eventos e devolve a mesma lista porém com os IPs dos eventos escrito "mascarado" no lugar.

Definição de tipos de dados:

A entrada será uma lista de eventos, que será representada pelo tipo composto `*List(Evento)*`. A saída será a mesma lista de eventos porém com os IPs dos eventos escrito "mascarado" no lugar, que será representada pelo tipo composto `*List(Evento)*`.

Especificação:

Recebe uma lista de eventos e devolve a mesma lista porém com os IPs dos eventos escrito "mascarado" no lugar.

Implementação:

A implementação está no arquivo *programa_completo.gleam* das linhas 346 à 351.

Exemplos:

Por conta dos exemplos não caberem no relatório, eles estão no mesmo arquivo da implementação das linhas 569 à 573

Revisão:

A implementação aplica a recursão estrutural diretamente sobre a lista de eventos, reconstruindo uma nova lista nó a nó sem alterar a estrutura de dados de entrada, respeitando estritamente o princípio da imutabilidade. A desestruturação por pattern matching (`[primeiro, ..resto]`) permite extrair cada evento individual e re-instanciá-lo substituindo apenas o campo `ip`. O caso base para lista vazia (`[] -> []`) encerra a construção de forma segura. A verificação via testes automatizados confirmou o mascaramento correto em listas vazias, unitárias e com múltiplos elementos.



Função que processe a estrutura autorreferente hierárquica:

Análise:

É necessário fazer uma função que encontre o setor mais perigoso em uma lista de setores

Definição de tipos de dados:

Uma lista de setores [List(Setor)], um ID do setor mais perigoso do tipo primitivo *Int* e a quantidade de eventos perigosos do tipo primitivo *Int*. A saída será o ID do setor mais perigoso, representado pelo tipo primitivo *Int*.

Especificação:

Recebe uma lista de setores, varre a lista de setores em busca do setor com a maior quantidade de eventos perigosos (severidade alta ou crítica), caso encontre, retorna o ID do setor mais perigoso, caso não encontre, retorna 0.

Implementação:

A implementação está no arquivo *programa_completo.gleam* das linhas 150 à 159

Exemplos:

```
// Check da funcao: busca_setor_mais_perigoso
pub fn busca_setor_perigoso_examples() {
  let evento1 = Evento(1,"Computador 1",Malware,Alta,3,EmAnalise)
  let evento2 = Evento(2,"Computador 2",AcessoSuspeito,Media,1,Desconhecido)
  let evento3 = Evento(3,"Computador 3",TentativaDeLogin,Critica,5,Resolvido)
  let evento4 = Evento(4,"Computador 4",Malware,Baixa,2,EmAnalise)

  let ativo1 = Ativo(1, "Ativo 1", [evento1, evento2])
  let ativo2 = Ativo(2, "Ativo 2", [evento1, evento3])
  let ativo3 = Ativo(3, "Ativo 3", [evento4])

  let setor1 = Setor(1, "Setor 1", [ativo1])
  let setor2 = Setor(2, "Setor 2", [ativo2])
  let setor3 = Setor(3, "Setor 3", [ativo3])

  check.eq(busca_setor_perigoso([setor1, setor2, setor3], 0, 0),2)
  check.eq(busca_setor_perigoso([setor1, setor2], 0, 0),2)
  check.eq(busca_setor_perigoso([setor1], 0, 0),1)
  check.eq(busca_setor_perigoso([], 0, 0),0)
}

// Check das auxiliares
pub fn acumula_eventos_perigosos_examples() {
  let evento1 =Evento(1,"Computador 1",Malware,Alta,3,EmAnalise)
  let evento2 =Evento(2,"Computador 2",AcessoSuspeito,Media,1,Desconhecido)
  let evento3 =Evento(3,"Computador 3",TentativaDeLogin,Critica,5,Resolvido)

  check.eq(acumula_eventos_perigosos([evento1, evento2, evento3],0),2)
  check.eq(acumula_eventos_perigosos([evento2],0),0)
  check.eq(acumula_eventos_perigosos([],0),0)
  check.eq(acumula_eventos_perigosos([evento1],0),1)
}

pub fn extrai_eventos_examples() {
  let evento1 =Evento(1,"Computador 1",Malware,Alta,3,EmAnalise)
  let evento2 =Evento(2,"Computador 2",AcessoSuspeito,Media,1,Desconhecido)
  let evento3 =Evento(3,"Computador 3",TentativaDeLogin,Critica,5,Resolvido)
  let evento4 =Evento(4,"Computador 4",Malware,Baixa,2,EmAnalise)

  let ativo1 =Ativo(1, "Ativo 1", [evento1, evento2])
  let ativo2 =Ativo(2, "Ativo 2", [evento3, evento4])
  let ativo3 =Ativo(3, "Ativo 3", [])

  check.eq(extrai_eventos([ativo1, ativo2], 0),2)
  check.eq(extrai_eventos([ativo1], 0),1)
  check.eq(extrai_eventos([ativo2], 0),1)
  check.eq(extrai_eventos([ativo3], 0),0)
  check.eq(extrai_eventos([], 0),0)
  check.eq(extrai_eventos([ativo1, ativo2, ativo3], 0),2)
}
```



Revisão:

Inicialmente, a função contaria com apenas uma função auxiliar, porém se fez necessário a construção da função *acumula_eventos_perigosos*. Desse modo a organização e legibilidade do código se consolidou de maneira agradável. A função pode ser melhorada evitando chamar *extrai_eventos* duas vezes numa mesma fase da recursão.